

if (condition) et ==, !=, <, > (opérateurs logiques de comparaison)

Description

L'instruction if ("si" en français), utilisée avec un opérateur logique de comparaison, permet de tester si une condition est vraie, par exemple si la mesure d'une entrée analogique est bien supérieure à une certaine valeur.

Syntaxe

Le format d'un test if est le suivant :

```
if (uneVariable > 50)
{
    // faire quelque chose
}
```

Dans cet exemple, le programme va tester si la variable `uneVariable` est supérieure à 50. Si c'est le cas, le programme va réaliser une action particulière. Autrement dit, si l'état du test entre les parenthèses est vrai, les instructions comprises entre les accolades sont exécutées. Sinon, le programme se poursuit sans exécuter ces instructions.

Les accolades peuvent être omises après une instruction if. Dans ce cas, la suite de la ligne (qui se termine par un point-virgule) devient la seule instruction de la condition. Tous les exemples suivants sont corrects :

```
if (x > 120)    digitalWrite(LEDpin, HIGH);

if (x > 120)
digitalWrite(LEDpin, HIGH);

if (x > 120){ digitalWrite(LEDpin, HIGH); }    // toutes ces formes sont
correctes
```

La condition évaluée à l'intérieur des parenthèses nécessite l'utilisation d'une ou plusieurs de ces opérateurs logiques de comparaison :

Opérateurs logiques de comparaison :

- `x == y` (x est égal à y)
- `x != y` (x est différent de y)
- `x < y` (x est inférieur à y)
- `x > y` (x est supérieur à y)
- `x <= y` (x est inférieur ou égal à y)
- `x >= y` (x est supérieur ou égal à y)

Attention :

Prenez garde à ne pas utiliser accidentellement le signe = unique (par exemple `if (x=10)`). Le signe égal unique est l'opérateur d'attribution d'une valeur, et fixe la valeur de x à 10 (autrement dit met la valeur 10 dans la variable x). Utilisez bien au lieu de cela le signe double égal `==` (c'est à dire `if (x==10)`), le `==` étant l'opérateur logique de comparaison, et qui test si x est bien égal à 10 ou non. Cette dernière condition (`x==10`) est vraie uniquement si x est égal à 10, alors que la première condition (`x=10`) sera toujours vraie.

C'est la raison pour laquelle le langage C (sur lequel est basé Arduino) évalue la condition (`x=10`) de la façon suivante : la valeur 10 est assignée à la variable 10 (se souvenir que l'opérateur = est l'opérateur d'attribution de valeur), et donc la variable x contient 10. Dès lors, l'instruction conditionnelle "if" teste la valeur 10, laquelle est toujours évaluée VRAIE. C'est pourquoi, l'instruction `if (x=10)` sera toujours considérée VRAIE, ce qui n'est pas le résultat voulu lorsque l'on utilise une instruction if. De plus, la variable x est mise à la valeur 10, ce qui n'est pas non plus l'action désirée.

if peut bien sûr être une partie d'une structure de contrôle de type branchement en utilisant l'instruction `if...else`.

if / else

Description

L'instruction if/else (si/sinon en français) permet un meilleur contrôle du déroulement du programme que la simple instruction if, en permettant de grouper plusieurs tests ensemble. Par exemple, une entrée analogique peut-être testée et une action réalisée si l'entrée est inférieure à 500, et une autre action réalisée si l'entrée est supérieure ou égale à 500. Le code ressemblera à cela :

```
if (brocheCinqEntree < 500)
{
    // action A
}
else
{
    // action B
}
```

else peut contenir un autre test if, et donc des tests multiples, mutuellement exclusifs peuvent être réalisés en même temps.

Chaque test sera réalisé après le suivant jusqu'à ce qu'un test VRAI soit rencontré. Quand une condition vraie est rencontrée, les instructions associées sont réalisées, puis le programme continue son exécution à la ligne suivant l'ensemble de la construction if/else. Si aucun test n'est VRAI, le bloc d'instructions par défaut else est exécuté, si il est présent, déterminant ainsi le comportement par défaut.

Noter qu'un bloc else if peut être utilisé avec ou sans bloc de conclusion else et vice versa. Un nombre illimité de branches else if est autorisé.

```
if (boutonPoussoir1 = 1)
{
    // faire l'action A
}
else if (boutonPoussoir2 = 1)
{
    // faire l'action B
}
else
{
    // faire l'action C
}
```

Une autre façon de réaliser un branchement de tests multiples mutuellement exclusifs est l'instruction switch case.

switch / case

Description

Tout comme l'instruction **if**, l'instruction **switch / case** ("commutateur de cas" en anglais) contrôle le déroulement des programmes. L'instruction **switch / case** permet au programmeur de construire une liste de "cas" (ou possibilités) à l'intérieur d'accolades. Le programme teste chaque cas avec la variable de test, et exécute le code si une correspondance (un test VRAI) est trouvé.

switch / case est légèrement plus flexible qu'une structure **if / else** en ce sens que le programmeur peut définir si la structure **switch** devra continuer à tester les cas sous la forme d'une liste, même après avoir trouvé une correspondance.

Si l'instruction **break** n'est pas trouvée après avoir exécuté le code d'une condition vraie, le programme continuera à tester les conditions restantes parmi les cas restants.

Si une instruction **break** est rencontrée, le cas fait sortir de la structure,

Paramètres

- **var** : variable dont vous voulez tester l'état
- **default** : si aucune autre condition vraie n'est rencontrée, le code default sera exécuté
- **break** : sans l'instruction **break**, l'instruction **switch** passera en revue toutes les instructions case à la recherche d'une condition vraie. Si une est trouvée, le code sera exécuté, même si ce n'est pas votre intention. L'instruction **break** indique à l'instruction **switch** d'arrêter de rechercher des conditions vraies, et fait sortir de la structure de choix.

Exemple / Syntaxe

```
switch (var) {  
  case 1:  
    //execute ca si 'var' = 1  
    break;  
  case 2:  
    //execute ca si 'var' = 2  
    break;  
  default:  
    // exécute ca si 'var' ne satisfait aucun des cas précédents  
    // default optionnel  
}
```

Conseils de programmation

On peut utiliser des caractères après les instructions case.

Les conditions de cas ne doivent pas nécessairement être listée de façon chronologique ou progressive.

Boucle while

Description

Les boucles **while** ("tant que" en anglais) bouclent sans fin, et indéfiniment, jusqu'à ce que la condition ou l'expression entre les parenthèses () devienne fausse. Quelque chose doit modifier la variable testée, sinon la boucle **while** ne se terminera jamais. Cela peut être dans votre code, soit une variable incrémentée, ou également une condition externe, soit le test d'un capteur.

Syntaxe

```
while(expression){ // tant que l'expression est vraie  
  
    // instructions à effectuer  
  
}
```

Paramètres

- **expression** : Une instruction (booléenne) C qui renvoie un résultat VRAI ou FAUX.

Exemple

```
var = 0;  
while(var < 200){ // tant que la variable est inférieur à 200  
  
    // fait quelque chose 200 fois de suite...  
  
    var++; // incrémente la variable  
  
}
```

random()

Description

L'instruction random() génère des nombres pseudo-aléatoires.

Syntaxe

```
long random(max)
long random(min, max)
```

Paramètres

- **min** : la limite inférieure pour la valeur aléatoire, inclusive (optionnel)
- **max** : la limite inférieure pour la valeur aléatoire, exclusive

Valeur renvoyée

un nombre aléatoire entre la valeur min et la valeur (max-1)

Exemple

```
void loop() {
  // affiche un nombre aléatoire entre 0 et 299
  randomNumber = random(300);
  Serial.println(randomNumber);

  // affiche un nombre aléatoire entre 10 et 19
  randomNumber = random(10, 20);
  Serial.println(randomNumber);

  delay(50); // pause de 50 ms
}
```

```
sketch_LEDAléatoire | Arduino 1.0
File Edit Sketch Tools Help

sketch_LEDAléatoire

/* Allumer une des trois couleurs
d'une diode RGB aléatoirement */

// Declaration des variables

int couleur = 0; // Nombre aléatoire qui va définir la couleur
int tempo = 1000; // définit le temps d'affichage de la couleur
int rouge = 13; // Assigne la couleur rouge sur le pin 13
int vert = 12; // Assigne la couleur verte sur le pin 12
int bleu = 11; // Assigne la couleur bleu sur le pin 11

void setup() { // Assigne les pins 'couleur' en sortie

pinMode(rouge, OUTPUT);
pinMode(vert, OUTPUT);
pinMode(bleu, OUTPUT);
}

void loop(){ // Début du programme en boucle

int couleur= random(3); // sélection d'un nombre aléatoire entre 0 et 2

switch (couleur){
  case 0: //Si 'couleur' = 0 alors la led s'allume en rouge
    digitalWrite(rouge, HIGH);
    delay(tempo);
    digitalWrite(rouge, LOW);
    break;

  case 1: //Si 'couleur' = 1 alors la led s'allume en vert
    digitalWrite(vert, HIGH);
    delay(tempo);
    digitalWrite(vert, LOW);
    break;

  case 2: //Si 'couleur' = 2 alors la led s'allume en bleu
    digitalWrite(bleu, HIGH);
    delay(tempo);
    digitalWrite(bleu, LOW);
    break;
}

}

// Fini

Done compiling.

Binary sketch size: 1926 bytes (of a 32256 byte maximum)
```


goto

Description

Fait sauter le déroulement du programme à un point précis du programme signalé par un label.

Syntaxe

```
label:  
  
goto label; // fait sauter le déroulement du programme au point label:
```

Conseil

L'utilisation de l'instruction **goto** est déconseillée en programmation en langage C, et certains auteurs de livres de programmation en langage C affirment que l'instruction **goto** n'est jamais nécessaire. Cependant, utilisé judicieusement, il peut simplifier certains programmes. La raison pour laquelle de nombreux programmeurs rechignent à utiliser l'instruction **goto** est qu'avec une utilisation non restreinte d'instructions **goto**, il est facile de créer des programmes qui ne pourront jamais être débogués.

Ceci étant dit, il y a des circonstances où l'instruction **goto** tombe à pic et simplifie l'écriture du programme. Une de ces situations est la sortie de boucles imbriquées, ou de blocs de condition [if](#) sur une condition précise.

Exemple

```
for(byte r = 0; r < 255; r++){  
    for(byte g = 255; g > -1; g--){  
        for(byte b = 0; b < 255; b++){  
            if (analogRead(0) > 250){ goto sortir;}  
            // plus d'instructions  
        }  
    }  
}  
  
sortir:
```

tone()

Description

Génère une onde carrée (onde symétrique avec "duty cycle" (niveau haut/période) à 50%) à la fréquence spécifiée (en Hertz (Hz), ou nombre de périodes par seconde) sur une broche. La durée peut être précisée, sinon l'impulsion continue jusqu'à l'appel de l'instruction noTone(). La broche peut être connectée à un buzzer piézoélectrique ou autre haut-parleur pour jouer des notes (les sons audibles s'étendent de 20Hz à 20 000Hz).

Une seule note peut être produite à la fois. Si une note est déjà jouée sur une autre broche, l'appel de la fonction tone() n'aura aucun effet (tant qu'une instruction noTone() n'aura pas eu lieu). Si la note est jouée sur la même broche, l'appel de la fonction tone() modifiera la fréquence jouée sur cette broche.

Syntaxe

```
tone(broche, frequence)
tone(broche, frequence, durée)
```

Paramètres

- **broche**: la broche sur laquelle la note est générée.
- **frequence**: la fréquence de la note produite, en hertz (Hz)
- **durée**: la durée de la note en millisecondes (optionnel)